

# executed\_residual

August 18, 2026

## 1 Residual-Population Triage Model — Alerts Outside the Fixed Rule List

### 1.1 Objective

The primary triage model (ASXAML-style) scores alerts generated by a fixed, pre-selected list of tunable rules. Any alert **not** on that list is currently routed straight to Review with no further differentiation — a blind spot in the overall system.

This notebook builds a **second, standalone triage model** scoped specifically to that residual population: alerts triggered by rules outside the fixed list (internal rules, catch-all/generic scenarios, or rules not yet productivity-tested). Its job is the same as the primary model’s — classify **STR-relevant** vs. **not STR-relevant** — but the feature set differs, since this population lacks the fixed-list scenario hit-counts that drove the primary model.

### 1.2 What this notebook covers

1. **Data preparation** — synthetic residual-population dataset (replace with a real extract), with the feature families discussed for this population: amount/velocity profiles, ratios, tenure, prior-alert history, STR-rate-by-rule, and structuring-adjacent signals (round-amount proximity).
2. **Exploratory checks** — class balance, missingness, feature distributions.
3. **Multiple models, each chosen for a distinct reason** — Logistic Regression, Naive Bayes, Random Forest, XGBoost, LightGBM — compared head-to-head on the same metrics (recall, precision, F-beta, reduced-FP-rate), following the ASXAML paper’s own multi-classifier comparison (their Experiment #5).
4. **Explainable results** — SHAP for the winning tree-based model, plus Logistic Regression coefficients as an independent statistical cross-check (does a transparent linear model agree with the tree model’s direction of effect?), same “two methods should agree” discipline used earlier for the primary model’s rule-direction analysis.
5. **Translation to thresholds** — brief closing section carrying the winning model’s SHAP output into deployable single-variable thresholds, consistent with the Form 1/2/3 approach used for the primary model.

Dependencies: pandas numpy scikit-learn xgboost lightgbm shap optuna matplotlib

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```

from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.metrics import (
    confusion_matrix, fbeta_score, recall_score, precision_score,
    accuracy_score, ConfusionMatrixDisplay, roc_curve, auc
)

import xgboost as xgb
import lightgbm as lgb
import shap
import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
BETA = 2.0    # recall-weighted F-beta, consistent with the primary triage model

```

## 1.3 1. Data preparation

### 1.3.1 1.1 Scope of this population

This model only ever sees alerts generated by rules **outside** the fixed, externally-tunable list — the population that currently auto-routes to Review with zero differentiation. Two labeling cautions carry over directly from the primary model’s build:

- **Exclude undispositioned alerts** (Pending New, Investigating) — same censoring logic as before; only fully dispositioned cases carry a usable label.
- **STR history features must respect the alert’s anchor date** — any feature built from “has this rule/customer had an STR before” must only look at STRs filed *before* this alert, or the feature leaks future information.

### 1.3.2 1.2 Feature families (synthetic generation mirrors these)

| Family                  | Examples                                                      |
|-------------------------|---------------------------------------------------------------|
| Amount/velocity profile | avg/std/max amount and txn count, 2-6 month windows           |
| Cross-window ratios     | current vs. historical behavior contrast                      |
| Network breadth         | distinct counterparties, days active                          |
| Tenure                  | account age, full-history flag                                |
| Prior alert/STR history | rule-level STR rate (volume-floored), customer prior STR flag |
| Structuring-adjacent    | % of transactions just under round/reporting thresholds       |

| Family | Examples     |
|--------|--------------|
| Static | account type |

We simulate this population with synthetic data below — replace `load_residual_alerts()` with a real extract before using this on production data.

```
[2]: def load_residual_alerts(n_customers=8000, seed=RANDOM_STATE):
    """
    STUB -- replace with a real extract of alerts triggered by rules OUTSIDE
    the fixed tunable list, joined to dispositions, e.g.:

        SELECT customer_id, alert_date, triggering_rule, disposition_outcome, ..
    ↪.
        FROM alerts a
        JOIN customer_profile_features f ON a.customer_id = f.customer_id
        AND f.as_of_date = a.alert_date
        WHERE a.triggering_rule NOT IN (SELECT rule_id FROM fixed_rule_list)
        AND a.case_status = 'Investigated'

    Simulates a customer-level feature table for this residual population, with
    a genuinely imbalanced STR label (~6%), consistent with the smaller yield
    typically seen in catch-all / not-yet-productivity-tested rule populations.
    """
    rng = np.random.default_rng(seed)

    # latent risk drives several correlated features + the label itself
    latent_risk = rng.beta(a=2.0, b=28, size=n_customers) # tuned for ~6% STR
    ↪rate, AUC ~0.77

    account_age_months = np.clip(rng.exponential(scale=36, size=n_customers),
    ↪1, 240)
    has_full_6m_history = (account_age_months >= 6).astype(int)

    avg_amt_6m = rng.lognormal(mean=8.0 + latent_risk * 1.5, sigma=1.0,
    ↪size=n_customers)
    avg_amt_2m = avg_amt_6m * (1 + latent_risk * rng.normal(1.5, 0.5,
    ↪n_customers).clip(0))
    # short-tenure accounts: no real 6m distinction available (per our
    ↪NaN-handling discussion)
    avg_amt_6m_reported = np.where(has_full_6m_history, avg_amt_6m, np.nan)

    avg_txncount_6m = rng.poisson(lam=5 + latent_risk * 40, size=n_customers).
    ↪astype(float)
    avg_txncount_2m = rng.poisson(lam=(5 + latent_risk * 40) * (1 +
    ↪latent_risk), size=n_customers).astype(float)
```

```

std_amt_6m = avg_amt_6m * rng.uniform(0.1, 0.6, n_customers) * (1 +
↳latent_risk)

distinct_counterparties_6m = rng.poisson(lam=2 + latent_risk * 25,
↳size=n_customers)
days_active_6m = np.clip(rng.poisson(lam=10 + latent_risk * 60,
↳size=n_customers), 0, 180)
max_daily_txn_count_6m = rng.poisson(lam=1 + latent_risk * 15,
↳size=n_customers)

months_since_last_alert = rng.exponential(scale=8, size=n_customers)

count_fixedlist_rule_hits_6m = rng.poisson(lam=0.3 + latent_risk * 3,
↳size=n_customers)
count_internal_rule_hits_6m = rng.poisson(lam=0.2 + latent_risk * 2,
↳size=n_customers)

# rule-level STR rate: the STR history of the SPECIFIC rule that fired this
↳alert
# (volume-floored in practice; here simulated as noisy but risk-correlated)
rule_str_rate_24m = np.clip(latent_risk * 0.5 + rng.normal(0.02, 0.02,
↳n_customers), 0, 1)

customer_prior_str_flag = (rng.random(n_customers) < np.clip(latent_risk *
↳4, 0, 0.4)).astype(int)

# structuring-adjacent: % of transactions just under a round reporting
↳threshold
pct_round_amount_txns_6m = np.clip(latent_risk * 0.6 + rng.normal(0.05, 0.
↳05, n_customers), 0, 1)

account_type = rng.choice(["internal", "external"], size=n_customers, p=[0.
↳65, 0.35])

# STR label -- combination of the strongest risk-correlated features + noise
logit = (
    -4.4
    + 10.0 * latent_risk
    + 1.6 * pct_round_amount_txns_6m
    + 1.3 * customer_prior_str_flag
    + 1.1 * rule_str_rate_24m
    + 0.18 * np.log1p(distinct_counterparties_6m)
    - 0.08 * np.log1p(account_age_months)
    + rng.normal(0, 0.35, n_customers)
)
prob_str = 1 / (1 + np.exp(-logit))

```

```

is_str = (rng.random(n_customers) < probab_str).astype(int)

df = pd.DataFrame({
    "customer_id": [f"C{i:06d}" for i in range(n_customers)],
    "avg_amt_2m": avg_amt_2m,
    "avg_amt_6m": avg_amt_6m_reported,
    "ratio_amt_2v6m": avg_amt_2m / np.where(has_full_6m_history,
↪avg_amt_6m, np.nan),
    "std_amt_6m": np.where(has_full_6m_history, std_amt_6m, np.nan),
    "avg_txncount_2m": avg_txncount_2m,
    "avg_txncount_6m": np.where(has_full_6m_history, avg_txncount_6m, np.
↪nan),
    "ratio_txncount_2v6m": avg_txncount_2m / np.where(has_full_6m_history,
↪avg_txncount_6m, np.nan),
    "distinct_counterparties_6m": distinct_counterparties_6m,
    "days_active_6m": days_active_6m,
    "max_daily_txn_count_6m": max_daily_txn_count_6m,
    "account_age_months": account_age_months,
    "has_full_6m_history": has_full_6m_history,
    "months_since_last_alert": months_since_last_alert,
    "count_fixedlist_rule_hits_6m": count_fixedlist_rule_hits_6m,
    "count_internal_rule_hits_6m": count_internal_rule_hits_6m,
    "rule_str_rate_24m": rule_str_rate_24m,
    "customer_prior_str_flag": customer_prior_str_flag,
    "pct_round_amount_txns_6m": pct_round_amount_txns_6m,
    "account_type": account_type,
    "is_str": is_str,
})

# replace any accidental inf from ratio divisions (per the inf/NaN
↪discussion)
df = df.replace([np.inf, -np.inf], np.nan)
return df

data = load_residual_alerts()
print(f"Residual population: {len(data):,} customers")
print(f"STR rate: {data['is_str'].mean():.2%} ({data['is_str'].sum():,}
↪positives)")
data.head()

```

Residual population: 8,000 customers

STR rate: 6.05% (484 positives)

/tmp/ipykernel\_515/1349770636.py:77: RuntimeWarning: divide by zero encountered in divide

```

"ratio_txncount_2v6m": avg_txncount_2m / np.where(has_full_6m_history,
avg_txncount_6m, np.nan),

```

```

[2]:  customer_id  avg_amt_2m  avg_amt_6m  ratio_amt_2v6m  std_amt_6m  \
0      C000000  5164.226828  4613.592797      1.119350  2472.757106
1      C000001  1165.163168  1113.382150      1.046508  273.975017
2      C000002  4508.335133      NaN      NaN      NaN
3      C000003   363.656568   338.489844      1.074350  187.726715
4      C000004  2986.805565  2643.720262      1.129774  1196.974679

      avg_txncount_2m  avg_txncount_6m  ratio_txncount_2v6m  \
0              8.0              6.0              1.333333
1              5.0              7.0              0.714286
2             13.0             NaN              NaN
3              8.0              7.0              1.142857
4              9.0              8.0              1.125000

      distinct_counterparties_6m  days_active_6m  ...  account_age_months  \
0              3              13  ...      22.407439
1              1              15  ...      25.649349
2              6              15  ...       5.718665
3              2              7  ...      88.128374
4              3              16  ...      39.800324

      has_full_6m_history  months_since_last_alert  count_fixedlist_rule_hits_6m  \
0              1              0.783765              0
1              1              2.166209              0
2              0              9.013098              0
3              1              5.894901              1
4              1              17.267790              0

      count_internal_rule_hits_6m  rule_str_rate_24m  customer_prior_str_flag  \
0              0              0.036631              0
1              0              0.077683              1
2              0              0.096710              0
3              0              0.062416              0
4              0              0.079612              1

      pct_round_amount_txns_6m  account_type  is_str
0              0.133095      internal      0
1              0.094126      external      0
2              0.184036      internal      0
3              0.180037      internal      0
4              0.126368      internal      0

```

[5 rows x 21 columns]

### 1.3.3 1.3 Encode categoricals, split features/label

`account_type` is the one categorical column (per the ASXML dataset description convention) — encode as a single binary flag rather than one-hot, keeping the feature set compact.

```
[3]: data["account_type_external"] = (data["account_type"] == "external").astype(int)

feature_cols = [c for c in data.columns if c not in ("customer_id",
↳ "account_type", "is_str")]
X = data[feature_cols].copy()
y = data["is_str"].copy()

print(f"Feature count: {len(feature_cols)}")
print(feature_cols)
```

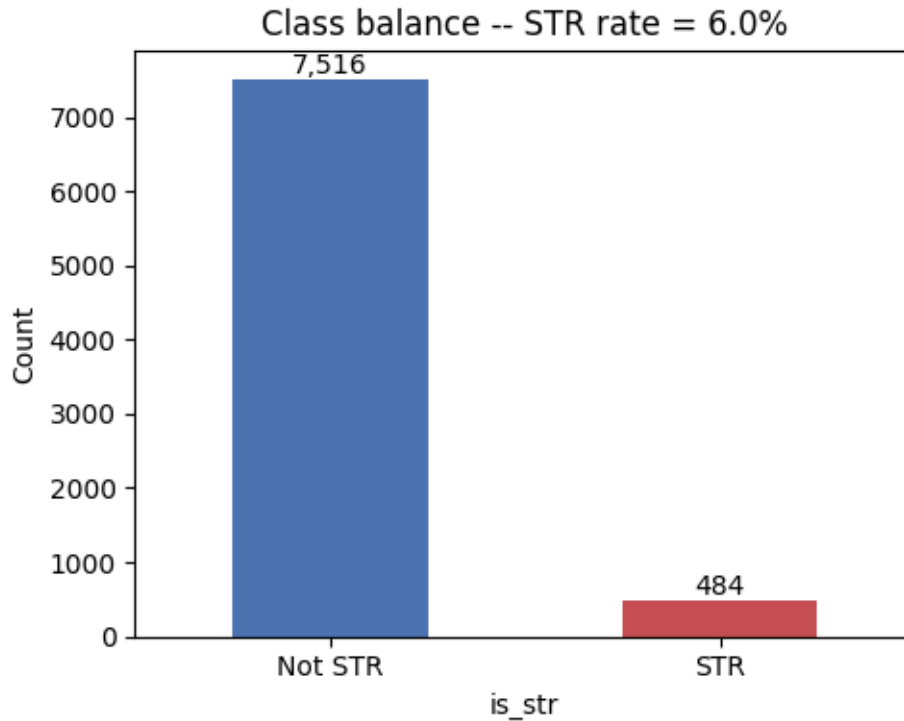
```
Feature count: 19
['avg_amt_2m', 'avg_amt_6m', 'ratio_amt_2v6m', 'std_amt_6m', 'avg_txncount_2m',
'avg_txncount_6m', 'ratio_txncount_2v6m', 'distinct_counterparties_6m',
'days_active_6m', 'max_daily_txn_count_6m', 'account_age_months',
'has_full_6m_history', 'months_since_last_alert',
'count_fixedlist_rule_hits_6m', 'count_internal_rule_hits_6m',
'rule_str_rate_24m', 'customer_prior_str_flag', 'pct_round_amount_txns_6m',
'account_type_external']
```

## 1.4 2. Exploratory checks

### 1.4.1 2.1 Class balance

Confirm the imbalance before choosing model settings — this determines whether `scale_pos_weight` / `class_weight` is necessary (per our earlier imbalance discussion) and how much to trust any single train/test split.

```
[4]: fig, ax = plt.subplots(figsize=(5, 4))
data["is_str"].value_counts().sort_index().plot(kind="bar", ax=ax,
↳ color=["#4c72b0", "#c44e52"])
ax.set_xticklabels(["Not STR", "STR"], rotation=0)
ax.set_ylabel("Count")
ax.set_title(f"Class balance -- STR rate = {data['is_str'].mean():.1%}")
for i, v in enumerate(data["is_str"].value_counts().sort_index()):
    ax.text(i, v, f"{v:,}", ha="center", va="bottom")
plt.tight_layout()
plt.show()
```



### 1.4.2 2.2 Missingness

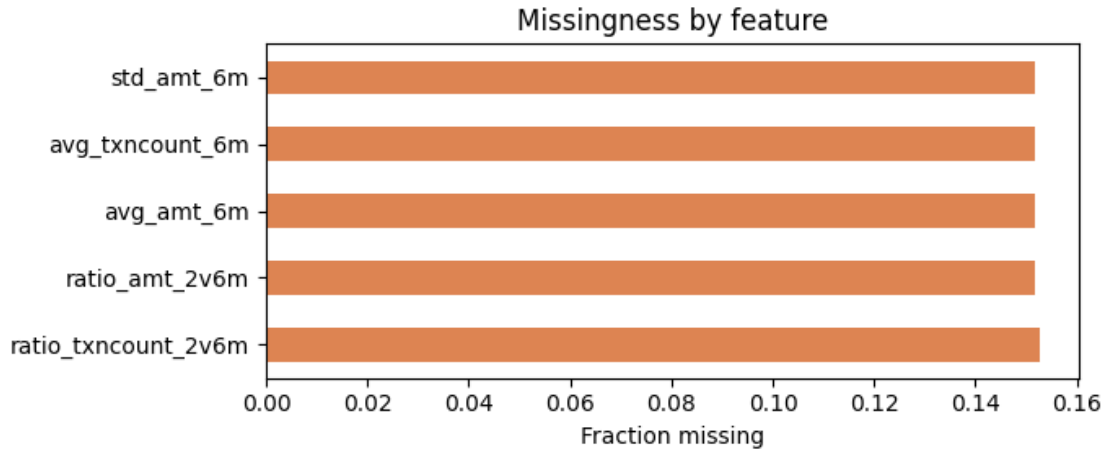
Per our earlier discussion: **NaN in the window features is meaningful** (insufficient tenure), not an error. Confirm the missingness pattern matches `has_full_6m_history` as expected, and note that tree-based models (RF, XGBoost, LightGBM) handle NaN natively, while Logistic Regression and Naive Bayes require imputation – handled per-model in Section 3.

```
[5]: missing_pct = X.isna().mean().sort_values(ascending=False)
missing_pct = missing_pct[missing_pct > 0]

fig, ax = plt.subplots(figsize=(7, 3))
missing_pct.plot(kind="barh", ax=ax, color="#dd8452")
ax.set_xlabel("Fraction missing")
ax.set_title("Missingness by feature")
plt.tight_layout()
plt.show()

print(f"Missingness matches short-tenure population: "
      f"{(1 - data['has_full_6m_history']).mean():.1%} of customers lack full_
↳6m history")
```





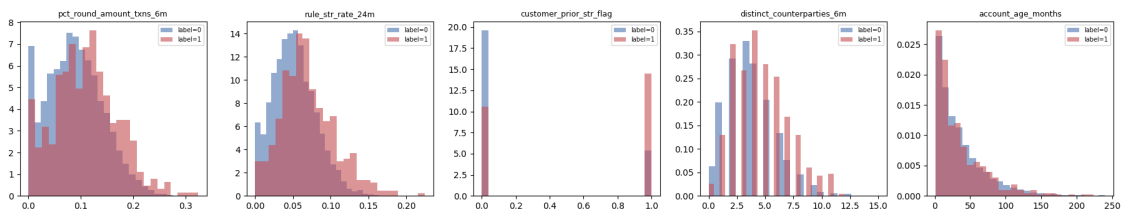
Missingness matches short-tenure population: 15.2% of customers lack full 6m history

### 1.4.3 2.3 Feature distributions by class

Quick visual check – the same “compare mean/distribution by label” logic used earlier for rule-direction analysis, now applied feature-by-feature to sanity check the synthetic data (and, on real data, to catch any feature whose direction is surprising before it reaches modeling).

```
[6]: check_features = ["pct_round_amount_txns_6m", "rule_str_rate_24m",
    ↪ "customer_prior_str_flag",
    ↪ "distinct_counterparties_6m", "account_age_months"]

fig, axes = plt.subplots(1, len(check_features), figsize=(18, 3.5))
for ax, feat in zip(axes, check_features):
    for label, color in [(0, "#4c72b0"), (1, "#c44e52")]:
        vals = X.loc[y == label, feat].dropna()
        ax.hist(vals, bins=25, alpha=0.6, density=True, color=color,
    ↪ label=f"label={label}")
    ax.set_title(feat, fontsize=9)
    ax.legend(fontsize=7)
plt.tight_layout()
plt.show()
```



### 1.5 3. Train/test split

Stratified split to preserve the (severe) class ratio in both sets. In a real deployment, prefer a **temporal** split (train on older alerts, test on more recent ones) per the discipline established for the primary triage model – kept stratified-random here only because the synthetic data has no real time-drift to respect.

```
[7]: X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.30, stratify=y, random_state=RANDOM_STATE
    )
    print(f"Train: {len(X_train):,} ({y_train.mean():.2%} STR)")
    print(f"Test: {len(X_test):,} ({y_test.mean():.2%} STR)")

    scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
    print(f"scale_pos_weight (for tree models): {scale_pos_weight:.1f}")
```

Train: 5,600 (6.05% STR)

Test: 2,400 (6.04% STR)

scale\_pos\_weight (for tree models): 15.5

### 1.6 4. Multiple models, and why each one is included

Following the ASXAML paper’s own multi-classifier comparison (their Experiment #5: SVM, RF, KNN, NB, DT), we compare **five models** here – each included for a distinct, deliberate reason rather than just to pad a comparison table:

| Model                      | Why it’s included                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Logistic Regression</b> | The interpretability baseline. Coefficients are directly readable (“a 1-unit increase in X changes log-odds by ”), giving a fully transparent cross-check against the tree models’ SHAP-derived directions (Section 6). If a tree model and logistic regression disagree on a feature’s direction, that’s a signal of a real interaction effect worth investigating, not noise to ignore.                                                                        |
| <b>Naive Bayes</b>         | A near-zero-cost baseline that assumes feature independence. Its main value here is diagnostic: if Naive Bayes performs close to the more sophisticated models, it suggests the features are largely independently informative (little interaction structure); if it performs much worse, that’s evidence the real signal lives in feature <i>combinations</i> – directly relevant to whether Section 6’s compound-rule extraction (Form 2) is worth the effort. |

| Model                | Why it's included                                                                                                                                                                                                                                                                                                                                     |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Random Forest</b> | A robust nonlinear baseline that handles the mixed feature types and missingness natively, is hard to overfit even with our modest positive count, and gives a second, independent feature-importance ranking to compare against XGBoost's.                                                                                                           |
| <b>XGBoost</b>       | The primary candidate, matching the primary triage model and both reference papers (ASXAML, Feedzai). Typically the strongest performer on structured, imbalanced tabular data of this kind.                                                                                                                                                          |
| <b>LightGBM</b>      | A speed/scalability comparison against XGBoost – histogram-based, natively fast on larger data, and (like XGBoost) handles missing values without imputation. Included to check whether the extra infrastructure of running two boosting libraries in production is ever justified by a real performance gap, or whether XGBoost alone is sufficient. |

**Metrics reported for every model:** recall, precision, F-beta ( $=2$ , recall-weighted per the primary model's convention), and reduced-false-positive-rate – never accuracy alone, for the reasons discussed with the primary model's confusion matrix.

#### 1.6.1 4.1 Preprocessing split – which models need imputation/scaling

Logistic Regression and Naive Bayes require complete, appropriately-scaled input. Random Forest, XGBoost, and LightGBM handle NaN natively and are scale-invariant (tree splits don't care about feature magnitude) – so we deliberately preprocess **twice**: once for the linear/probabilistic models, and pass the raw (NaN-preserving) data to the tree models, consistent with the “NaN is meaningful, don't paper over it” principle established earlier.

```
[8]: # -- version for Logistic Regression / Naive Bayes: imputed + scaled --
imputer = SimpleImputer(strategy="median")
scaler = StandardScaler()

X_train_imputed = pd.DataFrame(imputer.fit_transform(X_train), columns=X_train.
    ↪columns, index=X_train.index)
X_test_imputed = pd.DataFrame(imputer.transform(X_test), columns=X_test.
    ↪columns, index=X_test.index)

X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train_imputed),
    ↪columns=X_train.columns, index=X_train.index)
X_test_scaled = pd.DataFrame(scaler.transform(X_test_imputed), columns=X_test.
    ↪columns, index=X_test.index)
```

```
# -- version for tree models: raw, NaN preserved --
X_train_tree, X_test_tree = X_train.copy(), X_test.copy()

print("Preprocessing done: imputed+scaled set for LR/NB, raw NaN-preserving set_
↳for tree models")
```

Preprocessing done: imputed+scaled set for LR/NB, raw NaN-preserving set for tree models

## 1.6.2 4.2 Fit all five models

```
[9]: models = {}

# --- Logistic Regression ---
models["Logistic Regression"] = LogisticRegression(
    class_weight="balanced", max_iter=2000, random_state=RANDOM_STATE
).fit(X_train_scaled, y_train)

# --- Naive Bayes ---
models["Naive Bayes"] = GaussianNB().fit(X_train_scaled, y_train)

# --- Random Forest ---
models["Random Forest"] = RandomForestClassifier(
    n_estimators=300, max_depth=6, class_weight="balanced",
    random_state=RANDOM_STATE, n_jobs=-1
).fit(X_train_tree, y_train)

# --- XGBoost (lightweight Optuna tuning) ---
def xgb_objective(trial):
    params = {
        "objective": "binary:logistic",
        "eval_metric": "logloss",
        "max_depth": trial.suggest_int("max_depth", 2, 6),
        "eta": trial.suggest_float("eta", 0.01, 0.3, log=True),
        "subsample": trial.suggest_float("subsample", 0.6, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.6, 1.0),
        "scale_pos_weight": trial.suggest_float("scale_pos_weight",
↳scale_pos_weight * 0.5, scale_pos_weight * 1.5),
        "random_state": RANDOM_STATE,
    }
    cv = StratifiedKFold(n_splits=4, shuffle=True, random_state=RANDOM_STATE)
    scores = []
    for tr_idx, val_idx in cv.split(X_train_tree, y_train):
        m = xgb.XGBClassifier(**params, n_estimators=200, verbosity=0)
        m.fit(X_train_tree.iloc[tr_idx], y_train.iloc[tr_idx])
        preds = m.predict(X_train_tree.iloc[val_idx])
```

```

        scores.append(fbeta_score(y_train.iloc[val_idx], preds, beta=BETA,
↪zero_division=0))
        return float(np.mean(scores))

xgb_study = optuna.create_study(direction="maximize", sampler=optuna.samplers.
↪TPESampler(seed=RANDOM_STATE))
xgb_study.optimize(xgb_objective, n_trials=40, show_progress_bar=False)

best_xgb_params = dict(xgb_study.best_params)
best_xgb_params.update({"objective": "binary:logistic", "eval_metric":
↪"logloss", "random_state": RANDOM_STATE})
models["XGBoost"] = xgb.XGBClassifier(**best_xgb_params, n_estimators=300,
↪verbosity=0).fit(X_train_tree, y_train)

# --- LightGBM (lightweight Optuna tuning) ---
def lgb_objective(trial):
    params = {
        "objective": "binary",
        "num_leaves": trial.suggest_int("num_leaves", 8, 64),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3,
↪log=True),
        "min_child_samples": trial.suggest_int("min_child_samples", 10, 60),
        "scale_pos_weight": trial.suggest_float("scale_pos_weight",
↪scale_pos_weight * 0.5, scale_pos_weight * 1.5),
        "random_state": RANDOM_STATE,
        "verbosity": -1,
    }
    cv = StratifiedKFold(n_splits=4, shuffle=True, random_state=RANDOM_STATE)
    scores = []
    for tr_idx, val_idx in cv.split(X_train_tree, y_train):
        m = lgb.LGBMClassifier(**params, n_estimators=200)
        m.fit(X_train_tree.iloc[tr_idx], y_train.iloc[tr_idx])
        preds = m.predict(X_train_tree.iloc[val_idx])
        scores.append(fbeta_score(y_train.iloc[val_idx], preds, beta=BETA,
↪zero_division=0))
    return float(np.mean(scores))

lgb_study = optuna.create_study(direction="maximize", sampler=optuna.samplers.
↪TPESampler(seed=RANDOM_STATE))
lgb_study.optimize(lgb_objective, n_trials=40, show_progress_bar=False)

best_lgb_params = dict(lgb_study.best_params)
best_lgb_params.update({"objective": "binary", "random_state": RANDOM_STATE,
↪"verbosity": -1})
models["LightGBM"] = lgb.LGBMClassifier(**best_lgb_params, n_estimators=300).
↪fit(X_train_tree, y_train)

```

```

print("All 5 models fitted.")
print(f"Best XGBoost F-beta (CV): {xgb_study.best_value:.3f}")
print(f"Best LightGBM F-beta (CV): {lgb_study.best_value:.3f}")

```

All 5 models fitted.

Best XGBoost F-beta (CV): 0.364

Best LightGBM F-beta (CV): 0.354

### 1.6.3 4.3 Evaluate all five on the same held-out test set

Linear/probabilistic models score on the imputed+scaled test features; tree models score on the raw (NaN-preserving) test features – each model sees the representation it was actually trained on.

```

[10]: def evaluate_model(name, model, X_eval, y_eval, beta=BETA):
    preds = model.predict(X_eval)
    proba = model.predict_proba(X_eval)[: , 1]
    cm = confusion_matrix(y_eval, preds, labels=[1, 0])
    tp, fn = cm[0]
    fp, tn = cm[1]
    return {
        "model": name,
        "recall": recall_score(y_eval, preds, zero_division=0),
        "precision": precision_score(y_eval, preds, zero_division=0),
        "fbeta": fbeta_score(y_eval, preds, beta=beta, zero_division=0),
        "accuracy": accuracy_score(y_eval, preds),
        "reduced_fp_rate": tn / (tn + fp) if (tn + fp) > 0 else np.nan,
        "tp": tp, "fn": fn, "fp": fp, "tn": tn,
        "proba": proba,
    }

results = []
results.append(evaluate_model("Logistic Regression", models["Logistic_
    ↪Regression"], X_test_scaled, y_test))
results.append(evaluate_model("Naive Bayes", models["Naive Bayes"],
    ↪X_test_scaled, y_test))
results.append(evaluate_model("Random Forest", models["Random Forest"],
    ↪X_test_tree, y_test))
results.append(evaluate_model("XGBoost", models["XGBoost"], X_test_tree,
    ↪y_test))
results.append(evaluate_model("LightGBM", models["LightGBM"], X_test_tree,
    ↪y_test))

results_df = pd.DataFrame(results)[["model", "recall", "precision", "fbeta",
    ↪"accuracy", "reduced_fp_rate", "tp", "fn", "fp", "tn"]]

```

```
# naive "always predict majority class" baseline, per the
  ↳accuracy-is-misleading discussion
naive_accuracy = (y_test == 0).mean()
print(f"Naive baseline accuracy (always predict 'Not STR'): {naive_accuracy:.3f}
  ↳<- compare every model's accuracy against this")

results_df.round(3)
```

Naive baseline accuracy (always predict 'Not STR'): 0.940 <- compare every model's accuracy against this

```
[10]:
```

|   | model               | recall | precision | fbeta | accuracy | reduced_fp_rate \ |
|---|---------------------|--------|-----------|-------|----------|-------------------|
| 0 | Logistic Regression | 0.683  | 0.138     | 0.381 | 0.723    | 0.725             |
| 1 | Naive Bayes         | 0.290  | 0.200     | 0.266 | 0.887    | 0.925             |
| 2 | Random Forest       | 0.552  | 0.171     | 0.381 | 0.811    | 0.827             |
| 3 | XGBoost             | 0.648  | 0.160     | 0.403 | 0.774    | 0.782             |
| 4 | LightGBM            | 0.669  | 0.142     | 0.384 | 0.735    | 0.740             |

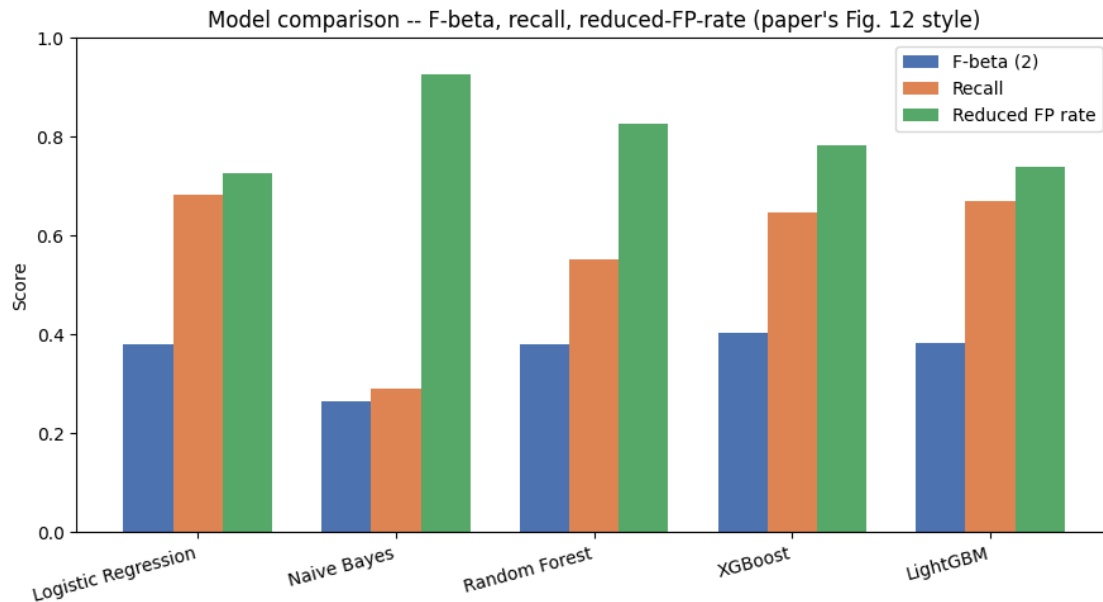
  

|   | tp | fn  | fp  | tn   |
|---|----|-----|-----|------|
| 0 | 99 | 46  | 619 | 1636 |
| 1 | 42 | 103 | 168 | 2087 |
| 2 | 80 | 65  | 389 | 1866 |
| 3 | 94 | 51  | 492 | 1763 |
| 4 | 97 | 48  | 587 | 1668 |

```
[11]: fig, ax = plt.subplots(figsize=(9, 5))
x = np.arange(len(results_df))
width = 0.25

ax.bar(x - width, results_df["fbeta"], width, label=f"F-beta ({BETA:.0f})",
  ↳color="#4c72b0")
ax.bar(x, results_df["recall"], width, label="Recall", color="#dd8452")
ax.bar(x + width, results_df["reduced_fp_rate"], width, label="Reduced FP
  ↳rate", color="#55a868")

ax.set_xticks(x)
ax.set_xticklabels(results_df["model"], rotation=15, ha="right")
ax.set_ylabel("Score")
ax.set_title("Model comparison -- F-beta, recall, reduced-FP-rate (paper's Fig.
  ↳12 style)")
ax.legend()
ax.set_ylim(0, 1)
plt.tight_layout()
plt.show()
```



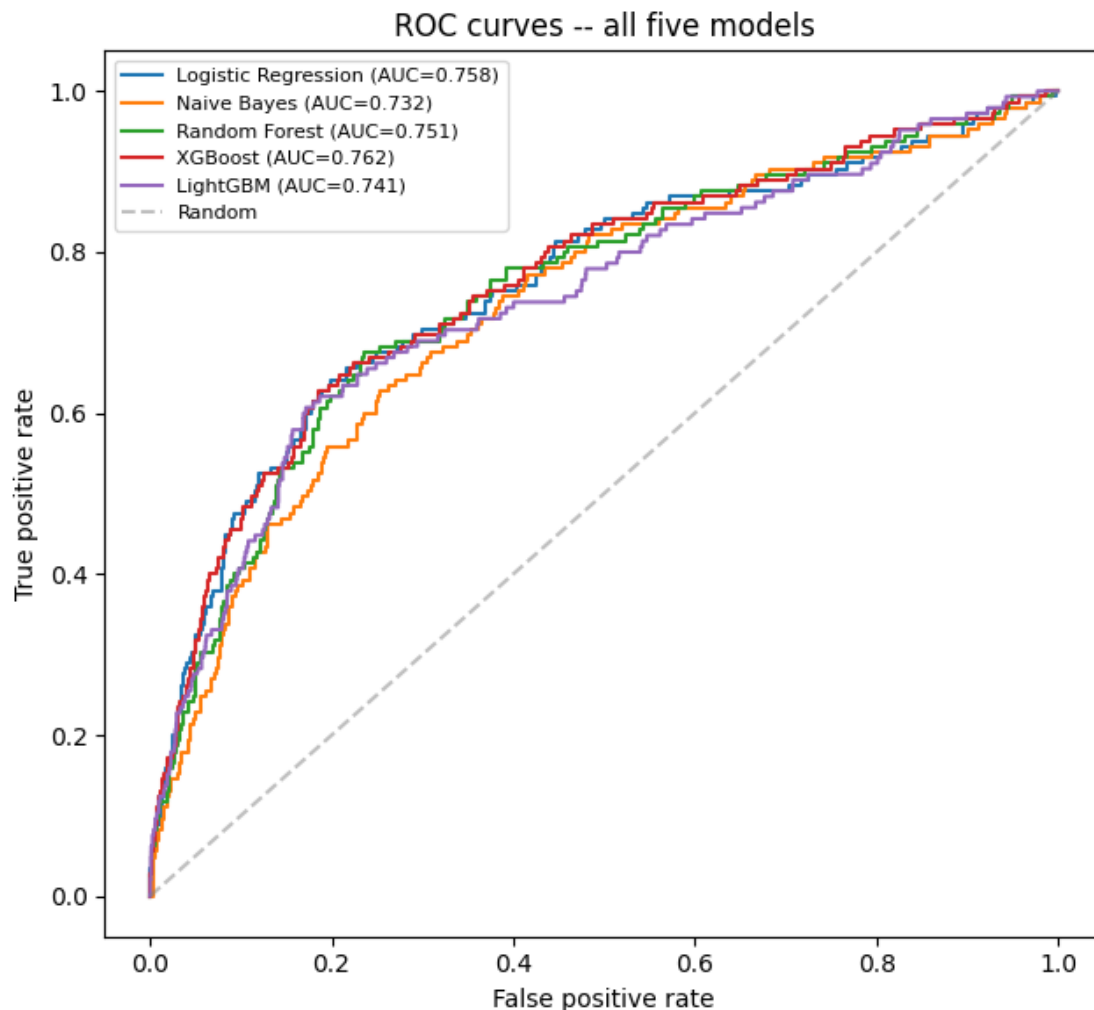
#### 1.6.4 4.4 ROC curves – a second view of the same comparison

Useful for seeing rank-ordering quality across the full threshold range, not just at the default 0.5 cutoff each model's `.predict()` uses above.

```
[12]: fig, ax = plt.subplots(figsize=(6.5, 6))
      for res in results:
          fpr, tpr, _ = roc_curve(y_test, res["proba"])
          roc_auc = auc(fpr, tpr)
          ax.plot(fpr, tpr, label=f"{res['model']} (AUC={roc_auc:.3f})")

      ax.plot([0, 1], [0, 1], linestyle="--", color="grey", alpha=0.5, label="Random")
      ax.set_xlabel("False positive rate")
      ax.set_ylabel("True positive rate")
      ax.set_title("ROC curves -- all five models")
      ax.legend(fontsize=8)
      plt.tight_layout()
      plt.show()
```





### 1.6.5 4.5 Selecting the winning model

Pick by F-beta (the metric aligned with the actual operating goal – recall- weighted, matching the primary triage model’s convention), not accuracy or AUC alone. Ties or near-ties should be broken by deployability considerations (tree models handle missing data and categorical-like features natively, without the imputation/scaling pipeline the linear models need in production).

```
[13]: best_row = results_df.sort_values("fbeta", ascending=False).iloc[0]
      best_model_name = best_row["model"]
      best_model = models[best_model_name]
      best_X_test = X_test_tree if best_model_name in ("Random Forest", "XGBoost",
      ↪ "LightGBM") else X_test_scaled
      best_X_train = X_train_tree if best_model_name in ("Random Forest", "XGBoost",
      ↪ "LightGBM") else X_train_scaled
```

```
print(f"Winning model by F-beta: {best_model_name} (F-beta={best_row['fbeta']:.3f}, recall={best_row['recall']:.3f})")
```

Winning model by F-beta: XGBoost (F-beta=0.403, recall=0.648)

## 1.7 5. Explainable results – statistical method (Logistic Regression coefficients)

Before turning to SHAP on the winning tree model, get an independent, fully-transparent read on feature direction from Logistic Regression’s coefficients. This mirrors the “two methods should agree” discipline from the primary model’s rule-direction analysis (mean-value comparison vs. SHAP sign) – here it’s coefficient sign vs. SHAP sign.

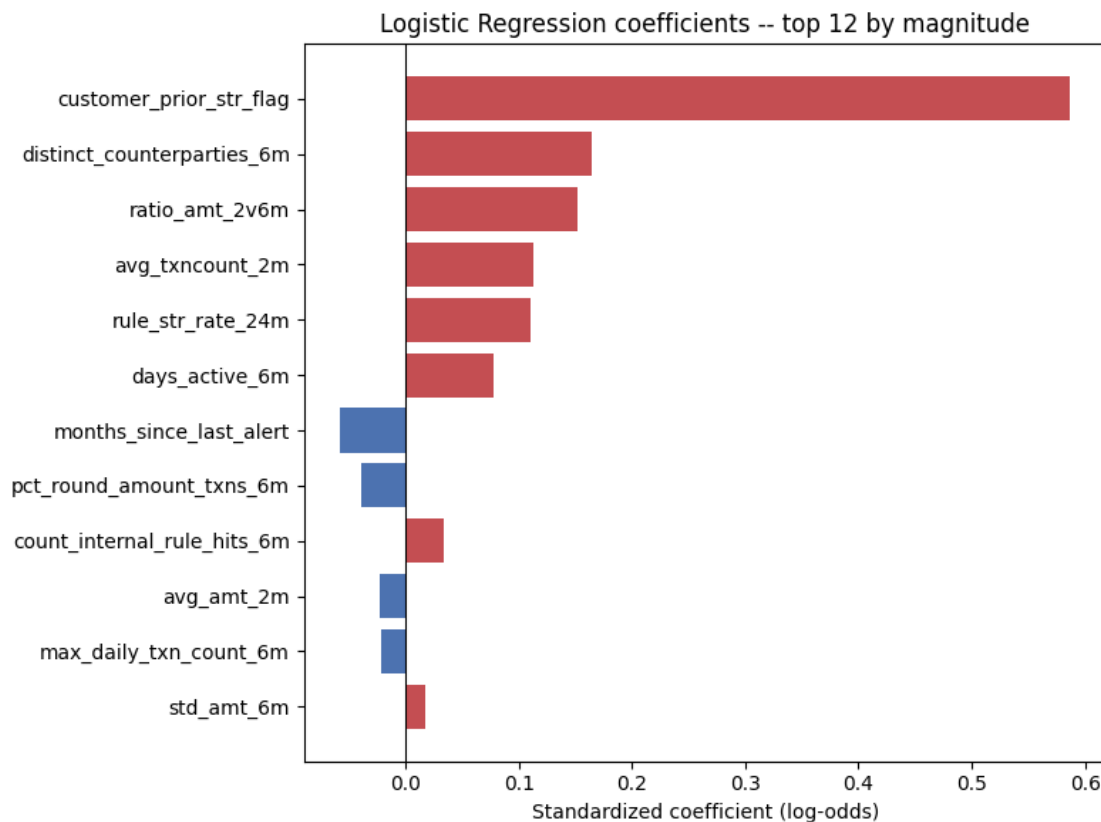
For a standardized logistic regression, each coefficient is directly interpretable: a positive coefficient means higher values of that feature increase the odds of STR; the magnitude (after standardization) is roughly comparable across features.

```
[14]: lr_model = models["Logistic Regression"]
coef_table = pd.DataFrame({
    "feature": X_train_scaled.columns,
    "coefficient": lr_model.coef_[0],
}).sort_values("coefficient", key=abs, ascending=False)

coef_table["direction"] = coef_table["coefficient"].apply(
    lambda c: "-> STR" if c > 0 else "-> Not STR"
)

fig, ax = plt.subplots(figsize=(8, 6))
top_coef = coef_table.head(12).iloc[::-1]
colors = ["#c44e52" if c > 0 else "#4c72b0" for c in top_coef["coefficient"]]
ax.barh(top_coef["feature"], top_coef["coefficient"], color=colors)
ax.axvline(0, color="black", linewidth=0.8)
ax.set_xlabel("Standardized coefficient (log-odds)")
ax.set_title("Logistic Regression coefficients -- top 12 by magnitude")
plt.tight_layout()
plt.show()

coef_table.head(12)
```



```
[14]:
```

|    | feature                     | coefficient | direction  |
|----|-----------------------------|-------------|------------|
| 16 | customer_prior_str_flag     | 0.587050    | -> STR     |
| 7  | distinct_counterparties_6m  | 0.164750    | -> STR     |
| 2  | ratio_amt_2v6m              | 0.152026    | -> STR     |
| 4  | avg_txncount_2m             | 0.112755    | -> STR     |
| 15 | rule_str_rate_24m           | 0.111069    | -> STR     |
| 8  | days_active_6m              | 0.077803    | -> STR     |
| 12 | months_since_last_alert     | -0.057397   | -> Not STR |
| 17 | pct_round_amount_txns_6m    | -0.038477   | -> Not STR |
| 14 | count_internal_rule_hits_6m | 0.034136    | -> STR     |
| 0  | avg_amt_2m                  | -0.023064   | -> Not STR |
| 9  | max_daily_txn_count_6m      | -0.021423   | -> Not STR |
| 3  | std_amt_6m                  | 0.017106    | -> STR     |

## 1.8 6. Explainable results – SHAP on the winning model

If the winning model is tree-based (Random Forest, XGBoost, or LightGBM), `TreeExplainer` gives exact, fast SHAP values. Compare the resulting direction per feature against the Logistic Regression coefficients above – agreement strengthens confidence in the signal; disagreement flags a feature whose real relationship is non-monotone or interaction-dependent (worth a dependence plot before trusting either single number).

```
[15]: if best_model_name in ("Random Forest", "XGBoost", "LightGBM"):
    explainer = shap.TreeExplainer(best_model)
    shap_values = explainer.shap_values(best_X_train)
    if isinstance(shap_values, list): # some RF/older-API outputs return a
    ↪list per class
        shap_values = shap_values[1]
    else:
        explainer = shap.LinearExplainer(best_model, best_X_train)
        shap_values = explainer.shap_values(best_X_train)

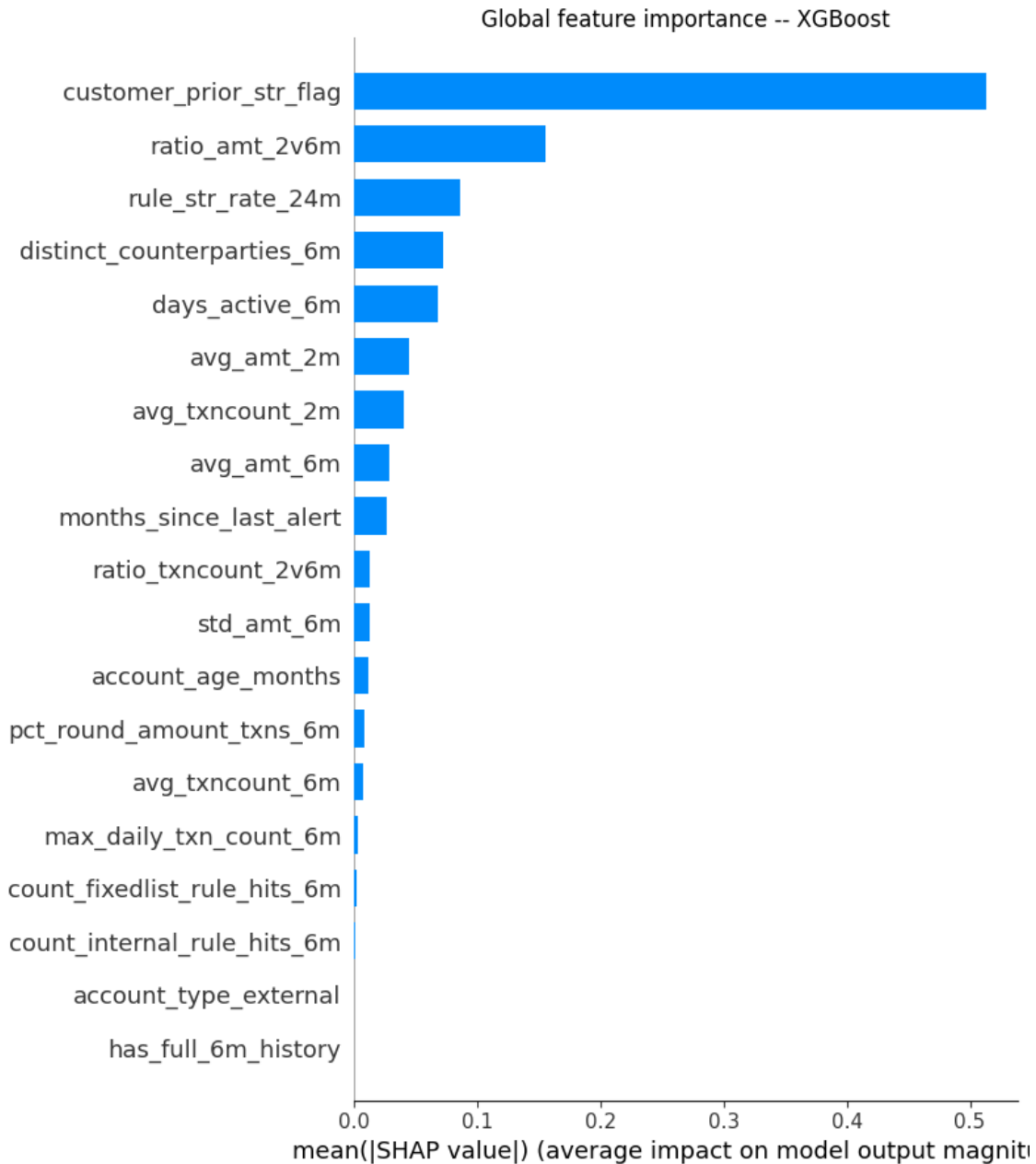
    print(f"SHAP values computed for winning model: {best_model_name},
    ↪shape={shap_values.shape}")
```

SHAP values computed for winning model: XGBoost, shape=(5600, 19)

```
[16]: shap.summary_plot(shap_values, best_X_train, plot_type="bar", show=False)
plt.title(f"Global feature importance -- {best_model_name}")
plt.tight_layout()
plt.show()
```

/tmp/ipykernel\_515/873886435.py:1: FutureWarning: The NumPy global RNG was seeded by calling `np.random.seed`. In a future version this function will no longer use the global RNG. Pass `rng` explicitly to opt-in to the new behaviour and silence this warning.

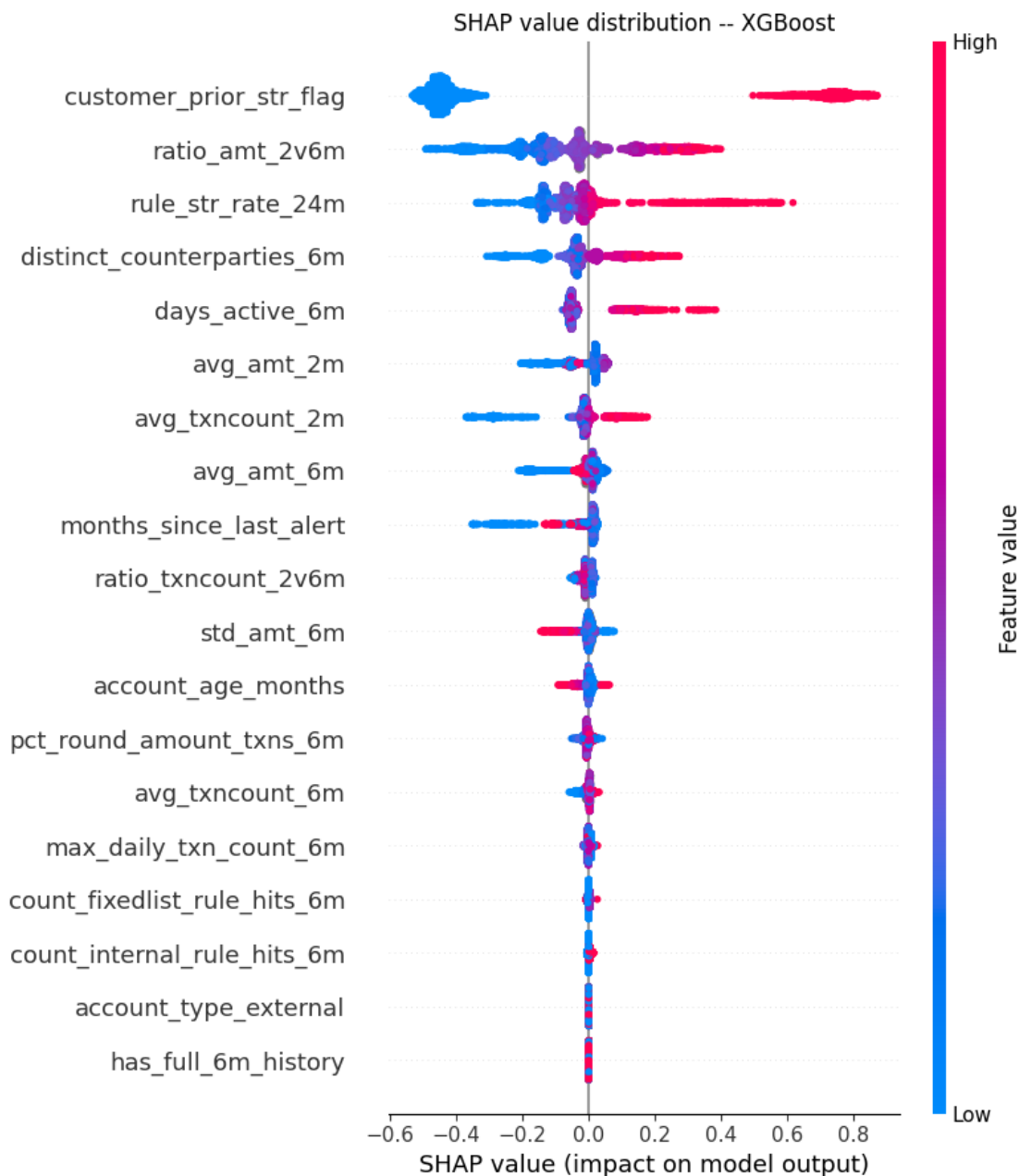
```
shap.summary_plot(shap_values, best_X_train, plot_type="bar", show=False)
```



```
[17]: shap.summary_plot(shap_values, best_X_train, show=False)
plt.title(f"SHAP value distribution -- {best_model_name}")
plt.tight_layout()
plt.show()
```

/tmp/ipykernel\_515/2687661890.py:1: FutureWarning: The NumPy global RNG was seeded by calling `np.random.seed`. In a future version this function will no longer use the global RNG. Pass `rng` explicitly to opt-in to the new behaviour and silence this warning.

```
shap.summary_plot(shap_values, best_X_train, show=False)
```



### 1.8.1 6.1 Cross-check: SHAP direction vs. Logistic Regression direction

The table below joins both methods' verdicts per feature. Rows where they **disagree** are flagged explicitly – these deserve a closer look (a dependence plot, or treating the feature as interaction-only rather than a simple threshold) before it goes into any deployable rule.

```
[18]: mean_shap = pd.Series(shap_values.mean(axis=0), index=best_X_train.columns)
shap_direction = mean_shap.apply(lambda v: "-> STR" if v > 0 else "-> Not STR")

cross_check = pd.DataFrame({
    "mean_shap": mean_shap,
    "shap_direction": shap_direction,
}).join(coef_table.set_index("feature")[["coefficient", "direction"]],
        how="left")
cross_check.columns = ["mean_shap", "shap_direction", "lr_coefficient",
        "lr_direction"]
cross_check["agree"] = cross_check["shap_direction"] ==
        cross_check["lr_direction"]

cross_check_sorted = cross_check.reindex(mean_shap.abs().
        sort_values(ascending=False).index)
print(f"Agreement rate across features: {cross_check_sorted['agree'].mean():.
        1%}")
cross_check_sorted
```

Agreement rate across features: 47.4%

```
[18]:
```

|                              | mean_shap | shap_direction | lr_coefficient | \ |
|------------------------------|-----------|----------------|----------------|---|
| customer_prior_str_flag      | -0.173755 | -> Not STR     | 0.587050       |   |
| ratio_amt_2v6m               | -0.053068 | -> Not STR     | 0.152026       |   |
| rule_str_rate_24m            | -0.034121 | -> Not STR     | 0.111069       |   |
| distinct_counterparties_6m   | -0.019501 | -> Not STR     | 0.164750       |   |
| avg_txncount_2m              | -0.018135 | -> Not STR     | 0.112755       |   |
| days_active_6m               | -0.015266 | -> Not STR     | 0.077803       |   |
| avg_amt_2m                   | -0.008722 | -> Not STR     | -0.023064      |   |
| avg_amt_6m                   | -0.007964 | -> Not STR     | -0.000202      |   |
| months_since_last_alert      | -0.007080 | -> Not STR     | -0.057397      |   |
| std_amt_6m                   | -0.005791 | -> Not STR     | 0.017106       |   |
| account_age_months           | -0.004822 | -> Not STR     | -0.003351      |   |
| pct_round_amount_txns_6m     | -0.004024 | -> Not STR     | -0.038477      |   |
| ratio_txncount_2v6m          | -0.003512 | -> Not STR     | -0.009877      |   |
| avg_txncount_6m              | -0.002923 | -> Not STR     | 0.004033       |   |
| max_daily_txn_count_6m       | -0.000698 | -> Not STR     | -0.021423      |   |
| count_fixedlist_rule_hits_6m | -0.000400 | -> Not STR     | -0.011406      |   |
| count_internal_rule_hits_6m  | -0.000105 | -> Not STR     | 0.034136       |   |
| has_full_6m_history          | 0.000000  | -> Not STR     | -0.007838      |   |
| account_type_external        | 0.000000  | -> Not STR     | 0.003700       |   |

|                            | lr_direction | agree |
|----------------------------|--------------|-------|
| customer_prior_str_flag    | -> STR       | False |
| ratio_amt_2v6m             | -> STR       | False |
| rule_str_rate_24m          | -> STR       | False |
| distinct_counterparties_6m | -> STR       | False |

|                              |            |       |
|------------------------------|------------|-------|
| avg_txncount_2m              | -> STR     | False |
| days_active_6m               | -> STR     | False |
| avg_amt_2m                   | -> Not STR | True  |
| avg_amt_6m                   | -> Not STR | True  |
| months_since_last_alert      | -> Not STR | True  |
| std_amt_6m                   | -> STR     | False |
| account_age_months           | -> Not STR | True  |
| pct_round_amount_txns_6m     | -> Not STR | True  |
| ratio_txncount_2v6m          | -> Not STR | True  |
| avg_txncount_6m              | -> STR     | False |
| max_daily_txn_count_6m       | -> Not STR | True  |
| count_fixedlist_rule_hits_6m | -> Not STR | True  |
| count_internal_rule_hits_6m  | -> STR     | False |
| has_full_6m_history          | -> Not STR | True  |
| account_type_external        | -> STR     | False |

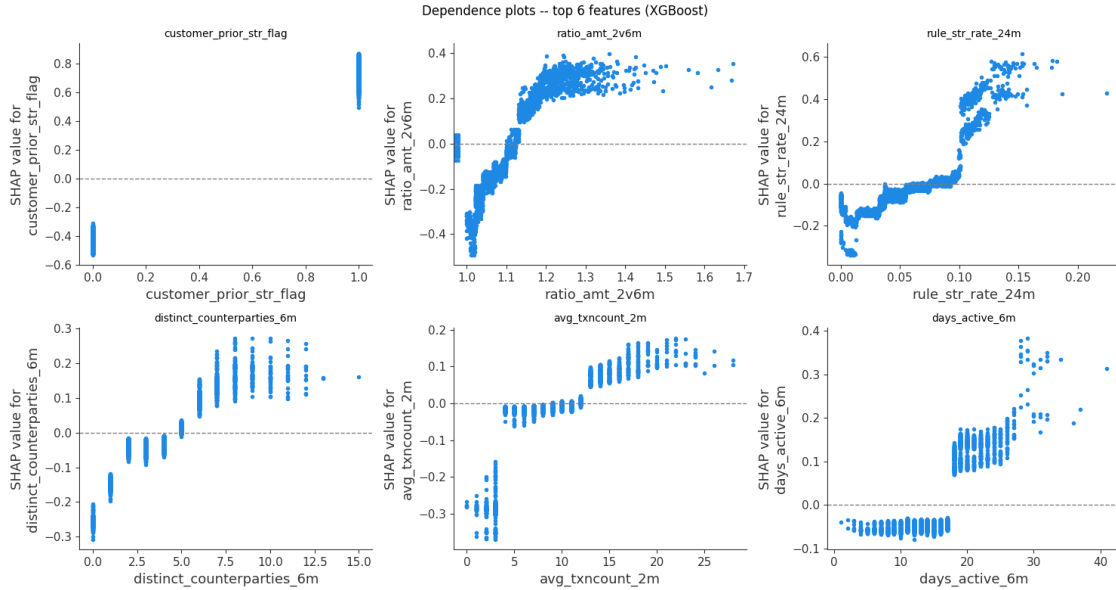
### 1.8.2 6.2 Dependence plots – top features, with candidate thresholds

Same Form-1 translation logic used for the primary model: where each curve crosses zero is a candidate single-variable threshold for this residual population's rules/scorecard.

```
[19]: top_features = mean_shap.abs().sort_values(ascending=False).head(6).index.
      ↪ tolist()

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for ax, feat in zip(axes.flat, top_features):
    shap.dependence_plot(feat, shap_values, best_X_train, ax=ax, show=False,
    ↪ interaction_index=None)
    ax.axhline(0, color="grey", linestyle="--", linewidth=1)
    ax.set_title(feat, fontsize=10)
plt.suptitle(f"Dependence plots -- top 6 features ({best_model_name})")
plt.tight_layout()
plt.show()
```





## 1.9 7. Summary and next steps

**Model comparison recap** (Section 4.3-4.4): five models spanning the interpretability-to-performance spectrum were compared on identical train/test splits and metrics. The winning model by F-beta is carried forward for explainability.

**Explainability recap** (Sections 5-6): two independent methods – Logistic Regression coefficients (a transparent statistical baseline) and SHAP on the winning tree model – were cross-checked against each other. Features where both methods agree on direction are strong, low-risk candidates for direct threshold/rule extraction; features where they disagree need the dependence plot's more detailed view before being trusted in a simple rule.

**Carried-forward outputs for the rule engine:** - Winning model identity and its test-set F-beta / recall / reduced-FP-rate - Per-feature direction table (Section 6.1), cross-validated by two methods - Candidate single-variable thresholds from the dependence plots (Section 6.2)

**Recommended next steps**, consistent with the primary model's build: 1. Validate the winning model's threshold on an **unsampled** population slice before setting any cutting value in production. 2. Extend Section 6.2 into the full Form 2 (compound rules) / Form 3 (scorecard) translation, exactly as built for the primary triage model. 3. Decide how this model's output interacts with the primary model's – e.g., a customer with an active fixed-list alert and a high-scoring residual-population alert may warrant escalation beyond what either model alone would suggest.

```
[20]: summary_table = results_df.copy()
      summary_table["selected"] = summary_table["model"] == best_model_name
      summary_table
```

```
[20]:
```

|   | model               | recall   | precision | fbeta    | accuracy \ |
|---|---------------------|----------|-----------|----------|------------|
| 0 | Logistic Regression | 0.682759 | 0.137883  | 0.381356 | 0.722917   |

|   |               |          |          |          |          |
|---|---------------|----------|----------|----------|----------|
| 1 | Naive Bayes   | 0.289655 | 0.200000 | 0.265823 | 0.887083 |
| 2 | Random Forest | 0.551724 | 0.170576 | 0.381316 | 0.810833 |
| 3 | XGBoost       | 0.648276 | 0.160410 | 0.403087 | 0.773750 |
| 4 | LightGBM      | 0.668966 | 0.141813 | 0.383703 | 0.735417 |

|   | reduced_fp_rate | tp | fn  | fp  | tn   | selected |
|---|-----------------|----|-----|-----|------|----------|
| 0 | 0.725499        | 99 | 46  | 619 | 1636 | False    |
| 1 | 0.925499        | 42 | 103 | 168 | 2087 | False    |
| 2 | 0.827494        | 80 | 65  | 389 | 1866 | False    |
| 3 | 0.781818        | 94 | 51  | 492 | 1763 | True     |
| 4 | 0.739690        | 97 | 48  | 587 | 1668 | False    |